

Case study - iviRA model

Xavier G.L.V. Pouwels

Introduction

This vignette shows how to use the functionalities of `pacheck` on the `iviRA` health economic model (Incerti et al. 2019).

Loading packages and data

```
devtools::load_all()
```

```
i Loading pacheck
```

```
Warning: replacing previous import 'glmnet::na.replace' by 'gtools::na.replace'
when loading 'pacheck'
```

```
Warning: replacing previous import 'boot::logit' by 'gtools::logit' when
loading 'pacheck'
```

```
Warning: replacing previous import 'boot::inv.logit' by 'gtools::inv.logit'
when loading 'pacheck'
```

```
Warning: replacing previous import 'assertthat::has_name' by 'tibble::has_name'
when loading 'pacheck'
```

```
set.seed(1234)
data("df_iviRA_pa")
```

Generating the probabilistic inputs and outputs

The source code to generate the model inputs and outputs of the **iviRA** cost-effectiveness model is available in the source code of the **pacheck** package on github (https://github.com/Xa4P/pacheck/blob/master/analysis/03-iviRA_data_generation.R). This script converts the probabilistic inputs and outputs list of a health economic analysis performed using the **iviRA** package to a dataframe of probabilistic inputs and outputs on which the **pacheck** tests can be performed.

The examples included in this appendix are performed on a subset of all model inputs and outputs to limit the size of the outputs provided by the **pacheck** tests. The functionalities of **pacheck** still work on a larger number of health economic (HE) model inputs since the tests included in **pacheck** are performed on the column-level of the dataframe of inputs and outputs. The number of columns/ parameters included in the check therefore does not affect the working of the tests implemented in **pacheck**.

Testing plausibility of model inputs and outputs

One way to test the plausibility of model inputs and outputs is simply to inspect the summary statistics of each variable. This can be done with the **pacheck** package using the `generate_sum_stats` function.

```
# Summary stats (10 first model inputs for long-term HAQ progression)
generate_sum_stats(df = df_iviRA_pa[, grep("r_haq_prog",
                                           names(df_iviRA_pa))])[1:10,]
```

	Parameter	Mean	SD	Percentile_2.5th	Percentile_97.5th
1	r_haq_prog.cdwards	0.031	0.003	0.026	0.036
2	r_haq_prog.abtvmx	-0.001	0.002	-0.004	0.002
3	r_haq_prog.abtscmx	-0.001	0.001	-0.004	0.002
4	r_haq_prog.adamx	-0.003	0.007	-0.018	0.011
5	r_haq_prog.ada	0.012	0.007	-0.003	0.026
6	r_haq_prog.adabiosbwmdmx	-0.003	0.008	-0.018	0.011
7	r_haq_prog.anamx	-0.001	0.001	-0.004	0.002
8	r_haq_prog.bct	-0.001	0.001	-0.004	0.002
9	r_haq_prog.bctmx	-0.001	0.001	-0.004	0.002
10	r_haq_prog.czp	-0.001	0.002	-0.004	0.002
	Minimum	Maximum	Median	Skewness	Kurtosis
1	0.022	0.038	0.031	-0.003	2.866
2	-0.005	0.004	-0.001	0.125	2.862
3	-0.006	0.004	-0.001	-0.043	3.034

4	-0.026	0.023	-0.003	0.028	2.881
5	-0.013	0.033	0.012	-0.125	3.155
6	-0.034	0.022	-0.003	-0.159	3.132
7	-0.006	0.004	-0.001	-0.023	2.753
8	-0.005	0.004	-0.001	-0.053	2.902
9	-0.005	0.004	-0.001	-0.026	2.878
10	-0.005	0.004	-0.001	0.032	2.844

```
# Specific checks
## Proportions & probabilities between 0 and 1
### p_discount parameters are discounts (rebates) on the prices of the medicines
v_vars_discount <- grep("p_discount", names(df_iviRA_pa), value = TRUE) # returns
# variable names containing p_discount
check_binary(c(v_vars_discount, "p_rebound"), df = df_iviRA_pa)
```

	Input	Negative_values	Values_above_1
1	p_discount.abtiv	None	None
2	p_discount.abtsc	None	None
3	p_discount.ada	None	None
4	p_discount.adabiosbwd	None	None
5	p_discount.ana	None	None
6	p_discount.bct	None	None
7	p_discount.czp	None	None
8	p_discount.etn	None	None
9	p_discount.etnbiosszs	None	None
10	p_discount.etnbiosykro	None	None
11	p_discount.gol	None	None
12	p_discount.hcl	None	None
13	p_discount.ifx	None	None
14	p_discount.ifxbiosqbtx	None	None
15	p_discount.cdmards	None	None
16	p_discount.rtx	None	None
17	p_discount.sar	None	None
18	p_discount.ssz	None	None
19	p_discount.tcz	None	None
20	p_discount.tof	None	None
21	p_discount.upa	None	None
22	p_rebound	None	None

```
## Costs, counts, OR positive
v_vars_n <- grep("n_", names(df_iviRA_pa), value = TRUE)
v_vars_c <- grep("c_", names(df_iviRA_pa), value = TRUE)
```

```
v_vars_c <- v_vars_c[!grep("inc_", v_vars_c)]
check_positive(c(v_vars_n, v_vars_c, "or_mort"), df = df_iviRA_pa)
```

	Input	Negative_values
1	n_hosp_days.haq_less0.5	None
2	n_hosp_days.haq0.5to1	None
3	n_hosp_days.haq1to1.5	None
4	n_hosp_days.haq1.5to2	None
5	n_hosp_days.haq2to2.5	None
6	n_hosp_days.haq2.5plus	None
7	c_hosp_pday.haq_less0.5	None
8	c_hosp_pday.haq0.5to1	None
9	c_hosp_pday.haq1to1.5	None
10	c_hosp_pday.haq1.5to2	None
11	c_hosp_pday.haq2to2.5	None
12	c_hosp_pday.haq2.5plus	None
13	c_si	None
14	c_prod_loss	None
15	or_mort	None

```
do_check(df = df_iviRA_pa,
  v_vars = c(v_vars_n, v_vars_c, "or_mort"),
  check = ~ .x >= 0,
  label_check = "positive") # alternative
```

\$check			
n_hosp_days.haq_less0.5	n_hosp_days.haq0.5to1	n_hosp_days.haq1to1.5	
TRUE	TRUE	TRUE	
n_hosp_days.haq1.5to2	n_hosp_days.haq2to2.5	n_hosp_days.haq2.5plus	
TRUE	TRUE	TRUE	
c_hosp_pday.haq_less0.5	c_hosp_pday.haq0.5to1	c_hosp_pday.haq1to1.5	
TRUE	TRUE	TRUE	
c_hosp_pday.haq1.5to2	c_hosp_pday.haq2to2.5	c_hosp_pday.haq2.5plus	
TRUE	TRUE	TRUE	
c_si	c_prod_loss	or_mort	
TRUE	TRUE	TRUE	

```
$messages
# A tibble: 1 x 2
  ok      message
<lg1> <glue>
1 TRUE all variables are positive
```

Metamodelling

In this section, we will perform metamodelling using two techniques: linear regression modelling (lm) and random forest (rf) using all input parameters of the probabilistic analysis data frame. Before performing metamodelling, we will standardise the inputs using their respective mean and standard deviation.

Fit

The metamodels were first developed on the full data sets, using all input parameters. The dependent variable of these metamodels is the “incremental QALY” between the two strategies we evaluated using the health economic model implemented in *iviRA*. Note that we did not perform variable selection, which could be performed to improve the metamodels’ fit. For the sake of simplicity, we will use the same input variables for the linear regression metamodel and the random forest metamodel.

In the `fit_rf_metamodel`, we set the `var_importance` argument to `TRUE` to be able to extract variable importance plots and for the random forest metamodel.

```
# Select parameters for metamodelling
v_x_vars <- names(df_iviRA_pa)
v_x_vars <- v_x_vars[-grep("t_qaly", v_x_vars)]
v_x_vars <- v_x_vars[-grep("t_cost", v_x_vars)]
v_x_vars <- v_x_vars[-grep("inc_cost", v_x_vars)]
v_x_vars <- v_x_vars[-grep("inc_qaly", v_x_vars)]
v_x_vars <- v_x_vars[-grep("p_discount", v_x_vars)]
v_x_vars <- v_x_vars[-grep("^c_", v_x_vars)]
#remove cost-related parameters

## standardisation
df_iviRA_mm <- df_iviRA_pa
df_iviRA_mm[, v_x_vars] <- as.data.frame(apply(df_iviRA_mm[, v_x_vars],
                                              2,
                                              function(col) {
                                                (col - mean(col)) / sd(col)}
                                              )
)

# Fit linear regression metamodel using all inputs contained in the dataframe
metamodel_lm <- fit_lm_metamodel(df = df_iviRA_mm,
                                y_var = "inc_qaly",
```

```

x_vars = v_x_vars)
summary(metamodel_lm$fit)

```

Call:

```
lm(formula = form, data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.72450	-0.17135	-0.00766	0.15943	1.00168

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.4154656	0.0078366	180.623	< 2e-16 ***
p_rebound	0.0063125	0.0079684	0.792	0.428445
or_mort	-0.0283213	0.0080916	-3.500	0.000487 ***
r_haq_prog.cdmards	-0.0065880	0.0080689	-0.816	0.414436
r_haq_prog.abtvmmtx	-0.0121438	0.0080121	-1.516	0.129932
r_haq_prog.abtscmtx	-0.0089875	0.0080476	-1.117	0.264364
r_haq_prog.adamtx	-0.0061606	0.0081067	-0.760	0.447476
r_haq_prog.ada	0.0046301	0.0079775	0.580	0.561786
r_haq_prog.adabiosbwddmtx	-0.0069485	0.0080592	-0.862	0.388806
r_haq_prog.anamtx	-0.0045046	0.0080735	-0.558	0.577011
r_haq_prog.bct	-0.0048906	0.0079885	-0.612	0.540545
r_haq_prog.bctmtx	0.0015498	0.0079867	0.194	0.846181
r_haq_prog.czp	-0.0019078	0.0081464	-0.234	0.814889
r_haq_prog.czpmmtx	0.0008299	0.0080857	0.103	0.918274
r_haq_prog.etn	0.0094061	0.0079997	1.176	0.239967
r_haq_prog.etnmmtx	0.0036882	0.0079802	0.462	0.644070
r_haq_prog.etnbiossszsmmtx	-0.0305528	0.0079837	-3.827	0.000138 ***
r_haq_prog.etnbiosykrommtx	0.0020836	0.0079898	0.261	0.794319
r_haq_prog.gol	-0.0043241	0.0080027	-0.540	0.589094
r_haq_prog.golmtx	0.0073567	0.0080304	0.916	0.359843
r_haq_prog.ifxmtx	-0.0090994	0.0079964	-1.138	0.255436
r_haq_prog.ifxbiosqbtmtx	-0.0036464	0.0080058	-0.455	0.648872
r_haq_prog.placebo	0.0047329	0.0080352	0.589	0.555987
r_haq_prog.nbt	0.0062929	0.0080774	0.779	0.436126
r_haq_prog.rtx	-0.0026376	0.0080485	-0.328	0.743203
r_haq_prog.rtxmtx	0.0062964	0.0080402	0.783	0.433756
r_haq_prog.sar	0.0065597	0.0080157	0.818	0.413359
r_haq_prog.sarmtx	0.0002034	0.0080653	0.025	0.979883
r_haq_prog.triple	-0.0028278	0.0080156	-0.353	0.724321

r_haq_prog.tcz	0.0050841	0.0080143	0.634	0.525985
r_haq_prog.tczmtx	0.0005645	0.0080657	0.070	0.944217
r_haq_prog.tofmtx	0.0143775	0.0080606	1.784	0.074796
r_haq_prog.tof	-0.0022986	0.0080392	-0.286	0.775002
r_haq_prog.upamtx	-0.0051709	0.0080526	-0.642	0.520934
n_hosp_days.haq_less0.5	-0.0064961	0.0080772	-0.804	0.421449
n_hosp_days.haq0.5to1	0.0071963	0.0080254	0.897	0.370113
n_hosp_days.haq1to1.5	0.0018147	0.0080528	0.225	0.821757
n_hosp_days.haq1.5to2	0.0016023	0.0082074	0.195	0.845260
n_hosp_days.haq2to2.5	-0.0002514	0.0079959	-0.031	0.974922
n_hosp_days.haq2.5plus	-0.0065000	0.0080240	-0.810	0.418104
u_model_wailoo.int	0.0062101	0.0080694	0.770	0.441737
u_model_wailoo.age	-0.0077827	0.0080091	-0.972	0.331436
u_model_wailoo.dis_dur	0.0025421	0.0080226	0.317	0.751410
u_model_wailoo.haq0	-0.0012470	0.0080794	-0.154	0.877374
u_model_wailoo.male	0.0042483	0.0080032	0.531	0.595660
u_model_wailoo.prev_dmards	0.0014110	0.0080122	0.176	0.860248
u_model_wailoo.haq	0.0022818	0.0080173	0.285	0.776008

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2478 on 953 degrees of freedom

Multiple R-squared: 0.04845, Adjusted R-squared: 0.002518

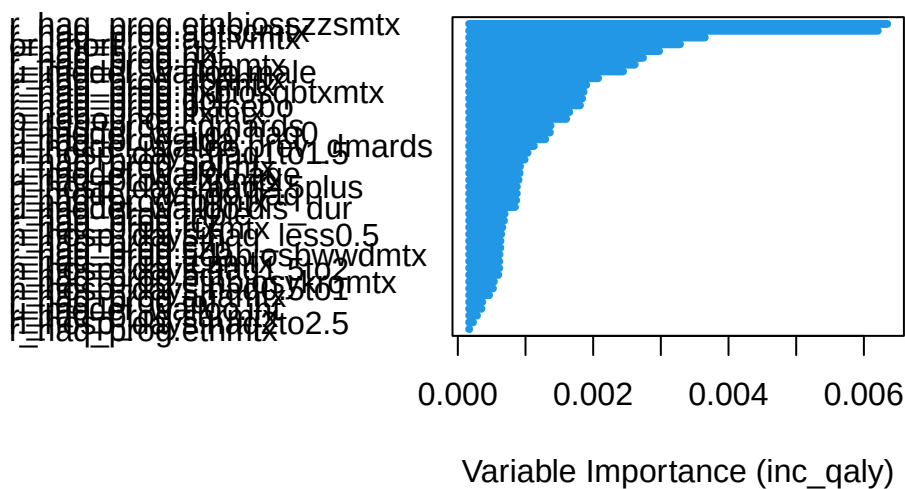
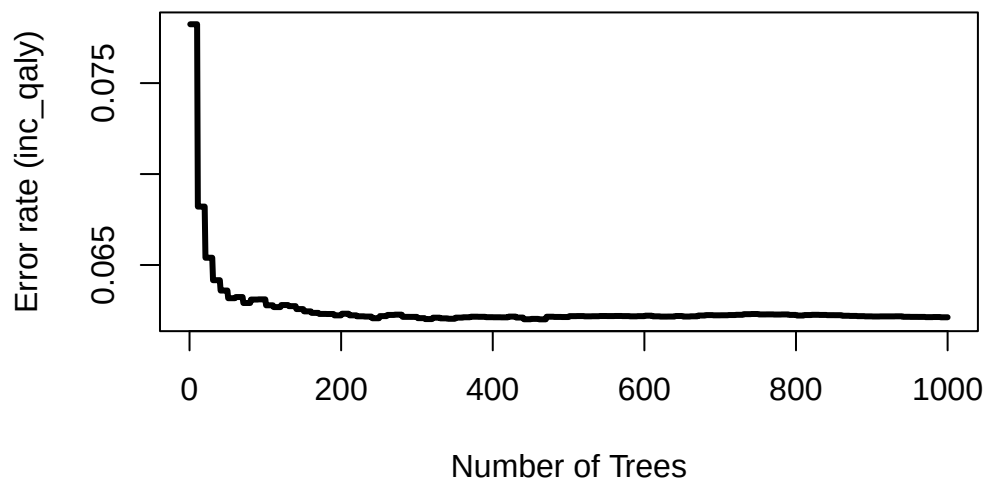
F-statistic: 1.055 on 46 and 953 DF, p-value: 0.3759

```

y <- df_iviRA_mm[, metamodel_lm$model_info$y_var]
predicted <- metamodel_lm$fit$fitted.values
r_squared_lm <- 1 - sum((y-predicted)^2)/sum((y-mean(y))^2)

metamodel_rf <- fit_rf_metamodel(df = df_iviRA_mm,
                                y_var = "inc_qaly",
                                x_vars = v_x_vars,
                                ntree = 1000,
                                var_importance = TRUE # plots variable importance
                                )

```



```
# R-squared for RF model because not exported by model summary
y <- metamodel_rf$fit$yvar
predicted <- metamodel_rf$fit$predicted
r_squared_rf <- 1 - sum((y-predicted)^2)/sum((y-mean(y))^2)
```

Interpretation

The R^2 of the linear regression metamodel, 0.048 is lower for this HE model than for the HE model implemented in the `cdx2cea` package. This can partly be explained by the patient-level nature of the HE model implemented in `iviRA` which may result in a less linear relationship between HE model inputs and outputs. In contrast, the R^2 of the random forest metamodel is 0.737, which is substantially higher than the R^2 of the linear regression metamodel. These two R^2 estimates indicate how well the both metamodel fit the training data, they however do not indicate how accurate the models are at predicting outcomes in data that is not used to train the metamodels. This is something we will investigate under the validation subheading.

Concerning the random forest metamodel, one can see that the error rate is not decreasing anymore once 1,000 are used in the random forest algorithm. This suggests that investigating the use of more trees within the algorithm may not lead to a further reduction of the error rate. Based on the values of the coefficients of the linear regression metamodels and the variable importance plots of the random forest metamodel (second plot), one can conclude that the model parameters relating to the progression of the HAQ score have the highest impact on the incremental QALYs.

Validation

Validation of the metamodel will be done using cross-validation with ten folds.

```
# Metamodels validation
validation_mm_lm <- validate_metamodel(
  model = metamodel_lm,
  method = 'cross_validation',
  folds = 10
)
validation_mm_rf <- validate_metamodel(
  model = metamodel_rf,
  method = 'cross_validation',
  folds = 10
)
knitr::kable(validation_mm_lm$stats_validation,
  caption = "Validation metrics of the linear regression metamodel.")
```

Table 1: Validation metrics of the linear regression metamodel.

Statistic	Value (method: cross-validation)
R-squared	-0.075
Mean absolute error	0.206
Mean relative error	0.152
Mean squared error	0.066

```
knitr::kable(validation_mm_rf$stats_validation,
  caption = "Validation metrics of the random forest metamodel with 1,000 trees.")
```

Table 2: Validation metrics of the random forest metamodel with 1,000 trees.

Statistic	Value (method: cross-validation)
R-squared	-0.020
Mean absolute error	0.201
Mean relative error	0.149
Mean squared error	0.062

Interpretation

Based on this validation step, both metamodels result in low R^2 estimates and high mean absolute and relative errors in the training sets. This may indicate overfitting of the metamodels

in the training set and a low generalisability of the metamodels. Hence, both models should actually not be used for making predictions. We will perform some predictions using these metamodels only for illustration purposes.

Prediction

The following code snippet shows how to make prediction using the fitted metamodels.

```
# Create dataframe containing the 10th and 90th percentile of the probabilistic values of the
df_inputs_pred <- data.frame(matrix(ncol = length(v_x_vars),
                                   nrow = 2,
                                   dimnames = list(c(),
                                                    v_x_vars))
                             )
df_inputs_pred[1, ] <- unname(apply(df_iviRA_mm[, v_x_vars],
                                   2,
                                   function(rows) {quantile(rows, probs = 0.1)}})
df_inputs_pred[2, ] <- unname(apply(df_iviRA_mm[, v_x_vars],
                                   2,
                                   function(rows) {quantile(rows, probs = 0.9)}})

# Predict
df_lm_preds <- predict_metamodel(model = metamodel_lm,
                                inputs = df_inputs_pred,
                                output_type = "long_df")
df_rf_preds <- predict_metamodel(model = metamodel_rf,
                                inputs = df_inputs_pred,
                                output_type = "long_df")

# Display
knitr::kable(df_lm_preds,
              caption = "Input parameter values and predictions using the linear regression m
```

Table 3: Input parameter values and predictions using the linear regression metamodel.

Name	Value 1	NA
p_rebound	-1.3662859	1.4206543
or_mort	-1.2655790	1.2752942
r_haq_prog.cdmards	-1.2933577	1.3070845
r_haq_prog.abtvmtn	-1.2113023	1.3444391
r_haq_prog.abtscmtx	-1.2406119	1.2661676

Name	Value 1	NA
r_haq_prog.adamtx	-1.2988305	1.2831231
r_haq_prog.ada	-1.2471242	1.1933453
r_haq_prog.adabiosbwwdmtx	-1.3081065	1.2580920
r_haq_prog.anamtx	-1.2662539	1.2925224
r_haq_prog.bct	-1.2992437	1.2578577
r_haq_prog.bctmtx	-1.2646205	1.2879133
r_haq_prog.czp	-1.2896426	1.2842670
r_haq_prog.czpmtx	-1.2913280	1.2905589
r_haq_prog.etn	-1.2769543	1.2822870
r_haq_prog.etnmtx	-1.2847053	1.2927767
r_haq_prog.etnbiosszzsmtx	-1.2928860	1.3267779
r_haq_prog.etnbiosykromtx	-1.2980219	1.2642812
r_haq_prog.gol	-1.2394650	1.3364172
r_haq_prog.golmtx	-1.3433478	1.2482804
r_haq_prog.ifxmtx	-1.2440296	1.2297170
r_haq_prog.ifxbiosqbttxmtx	-1.2669866	1.2861282
r_haq_prog.placebo	-1.2862516	1.2753708
r_haq_prog.nbt	-1.2832023	1.2112680
r_haq_prog.rtx	-1.3240859	1.3221409
r_haq_prog.rtxmtx	-1.2996200	1.3339424
r_haq_prog.sar	-1.3278804	1.3135008
r_haq_prog.sarmtx	-1.2696626	1.2945709
r_haq_prog.triple	-1.2463134	1.2946604
r_haq_prog.tcz	-1.2781745	1.3245868
r_haq_prog.tczmtx	-1.2919564	1.3194569
r_haq_prog.tofmtx	-1.2570010	1.3615963
r_haq_prog.tof	-1.3084609	1.2793805
r_haq_prog.upamtx	-1.2938237	1.2702892
n_hosp_days.haq_less0.5	-0.5051284	0.9921110
n_hosp_days.haq0.5to1	-0.2418955	0.2865862
n_hosp_days.haq1to1.5	-0.9054043	1.4112008
n_hosp_days.haq1.5to2	-1.0183669	1.3004630
n_hosp_days.haq2to2.5	-1.1758497	1.2863206
n_hosp_days.haq2.5plus	-1.2148175	1.2284620
u_model_wailoo.int	-1.3129443	1.2844062
u_model_wailoo.age	-1.2157838	1.2317669
u_model_wailoo.dis_dur	-1.2820319	1.1979991
u_model_wailoo.haq0	-1.3017471	1.3223054
u_model_wailoo.male	-1.3078176	1.1859927
u_model_wailoo.prev_dmards	-1.2944664	1.2591622
u_model_wailoo.haq	-1.1943372	1.3017535

Name	Value 1	NA
predictions	1.4874550	1.3388318

```
knitr::kable(df_rf_preds,
              caption = "Input parameter values and predictions using the random forest metamodel")
```

Table 4: Input parameter values and predictions using the random forest metamodel.

Name	Value 1	NA
p_rebound	-1.3662859	1.4206543
or_mort	-1.2655790	1.2752942
r_haq_prog.cdmards	-1.2933577	1.3070845
r_haq_prog.abtvmx	-1.2113023	1.3444391
r_haq_prog.abtscmtx	-1.2406119	1.2661676
r_haq_prog.adamtx	-1.2988305	1.2831231
r_haq_prog.ada	-1.2471242	1.1933453
r_haq_prog.adabiosbwmdmx	-1.3081065	1.2580920
r_haq_prog.anamtx	-1.2662539	1.2925224
r_haq_prog.bct	-1.2992437	1.2578577
r_haq_prog.bctmtx	-1.2646205	1.2879133
r_haq_prog.czp	-1.2896426	1.2842670
r_haq_prog.czpmx	-1.2913280	1.2905589
r_haq_prog.etn	-1.2769543	1.2822870
r_haq_prog.etnmtx	-1.2847053	1.2927767
r_haq_prog.etnbiosszsmtx	-1.2928860	1.3267779
r_haq_prog.etnbiosykromtx	-1.2980219	1.2642812
r_haq_prog.gol	-1.2394650	1.3364172
r_haq_prog.golmtx	-1.3433478	1.2482804
r_haq_prog.ifxmtx	-1.2440296	1.2297170
r_haq_prog.ifxbiosqbtvmx	-1.2669866	1.2861282
r_haq_prog.placebo	-1.2862516	1.2753708
r_haq_prog.nbt	-1.2832023	1.2112680
r_haq_prog.rtx	-1.3240859	1.3221409
r_haq_prog.rtxmtx	-1.2996200	1.3339424
r_haq_prog.sar	-1.3278804	1.3135008
r_haq_prog.sarmtx	-1.2696626	1.2945709
r_haq_prog.triple	-1.2463134	1.2946604
r_haq_prog.tcz	-1.2781745	1.3245868
r_haq_prog.tczmtx	-1.2919564	1.3194569
r_haq_prog.tofmtx	-1.2570010	1.3615963

Name	Value 1	NA
r_haq_prog.tof	-1.3084609	1.2793805
r_haq_prog.upamtx	-1.2938237	1.2702892
n_hosp_days.haq_less0.5	-0.5051284	0.9921110
n_hosp_days.haq0.5to1	-0.2418955	0.2865862
n_hosp_days.haq1to1.5	-0.9054043	1.4112008
n_hosp_days.haq1.5to2	-1.0183669	1.3004630
n_hosp_days.haq2to2.5	-1.1758497	1.2863206
n_hosp_days.haq2.5plus	-1.2148175	1.2284620
u_model_wailoo.int	-1.3129443	1.2844062
u_model_wailoo.age	-1.2157838	1.2317669
u_model_wailoo.dis_dur	-1.2820319	1.1979991
u_model_wailoo.haq0	-1.3017471	1.3223054
u_model_wailoo.male	-1.3078176	1.1859927
u_model_wailoo.prev_dmards	-1.2944664	1.2591622
u_model_wailoo.haq	-1.1943372	1.3017535
predictions	1.4823220	1.4202506

References

Incerti, Devin, Jeffrey R. Curtis, Jason Shafrin, Darius N. Lakdawalla, and Jeroen P. Jansen. 2019. “A Flexible Open-Source Decision Model for Value Assessment of Biologic Treatment for Rheumatoid Arthritis.” *PharmacoEconomics* 37 (6): 829–43. <https://doi.org/10.1007/s40273-018-00765-2>.